Data Structures

BS (CS) _SP_2026

Assignment 3
+
Lab_11 Tasks

Learning Objectives:

1. Trees

Submission Instructions

1. Take screenshots of the test case results. Name them as *T<TaskNo>*e.g., **T1.PNG**
2. Name your files using the format *T<TaskNo>*, e.g., **T1.h**.
3. Compress all .cpp, .h and .png files into a .zip file, and name it as *ROLLNO_SEC_LAB<NO>/ ROLLNO_SEC_A<NO>* e.g. i25XXXX_A_LAB11/ i25XXXX_A_A03.
4. Now you have to submit this zipped file on Google Classroom.
5. After uploading your file, recheck it to ensure that all files are correct and that no file is missing or corrupted.
6. Failure to follow the above-mentioned submission instructions will result in zero marks.
7. Any plagiarism in the lab task will result in zero marks for the entire category.

Tasks

DIGITAL CONTENT MANAGEMENT SYSTEM

Problem Statement

A large digital media organization manages thousands of content items such as articles, scripts, blogs, and research drafts. Each content item undergoes multiple revisions over time as writers continuously edit and improve their work.

To efficiently manage this process, the organization requires a system that can:

- Maintain a complete history of all revisions for each content item
- Provide fast search capabilities across content titles and stored text
- Support system snapshots for backup and restoration
- Analyze how revision structures evolve over time

Your task is to design and implement an Advanced Digital Content Revision System using Binary Search Trees (BST).

All components must be implemented using dynamic node-based data structures only:

- No STL containers
- No static arrays

The system must compute and analyze the balance factor of revision trees to evaluate whether the structure remains balanced or becomes skewed over time.

System Design (Physical / Logical Model)

1. **Content Catalog (Global Index)**
    - Implement a BST storing all content items
    - **Key:** Content Title
    - **Value:** ContentMetadata
2. **Per-Content Revision History**
    - Each content item maintains its own BST
    - **Key:** Timestamp (Timestamps must be unique and monotonically increasing per content item.)
    - **Value:** Revision node
3. **Balance Factor Analysis**
   Each revision tree must support:

   Balance Factor = height(left subtree) - height(right subtree)

    - Used for analysis only
    - No rotations or self-balancing required
4. **Data Structure Constraints**
    - Only dynamic node-based structures allowed
    - All trees and lists must be implemented manually

Class Design

```
class ContentManagementSystem {
public:
    ContentMetadata* contentRoot;
    Snapshot* snapshots;

    void createContent(string title, string text);
    void addRevision(string title, string newText);
```

```
    void deleteContent(string title);

    void analyzeContentHistory(string title);
    void systemAnalysis();

    void createSnapshot(string snapshotName);
    void restoreSnapshot(string snapshotName);
    void listSnapshots();

    void printRevisions(string title);
    void exportContent(string title, int revisionNo, const string& outFileName);

    SearchResultNode* searchContentTitles(string pattern);

    SearchResultNode* searchText(string keyword);

    Revision* getRevision(string title, long timestamp);
    ContentMetadata* findContent(string title);

    void saveToDisk(string filename);
    void loadFromFile(string filePath);
    long getLatestTimestamp(string title);
    int getBalanceFactor(Revision* node);


};
```

```
class Revision {
public:
    string title;
    string text;
    int size;
    long timestamp;
    int revisionNumber;

    Revision* left;
    Revision* right;

    int height;   // used for balance factor calculation
};
```

```
class ContentMetadata {
public:
```

```
    string title;

    Revision* currentRevision;
    Revision* revisionRoot;

    int totalRevisions;
    long lastModified;
    int totalSize;

    ContentMetadata* left;
    ContentMetadata* right;
};
```

```
class SearchResultNode {
public:
    string title;
    int revisionNumber;
    SearchResultNode* next;
};
```

```
class Snapshot {
public:
    string name;
    ContentMetadata* rootCopy;
    Snapshot* next;
};
```

Core Functional Requirements

- **Content Operations**
  a. **Create Content**

     void createContent(string title, string text);

     i. Create a new content item
     ii. Store its first revision
     iii. Insert into BST
     iv. If content with the same title already exists, the system must ignore the request.

  b. **Add Revision**

void addRevision(string title, string newText);

i. Create new revision
ii. Insert into revision BST
iii. Update metadata

c. **Retrieve Revision**

Revision* getRevision(string title, long timestamp);

Return:

i. Exact match OR
ii. Closest earlier revision
iii. If no earlier revision exists, return nullptr.

d. **Delete Content**

void deleteContent(string title);

i. Remove content from BST
ii. Free all associated memory

- **Search Operations**
  a. **Search Content Titles**

  SearchResultNode* searchContentTitles(string pattern);

  Supports:

  i. Partial matching
  ii. Case-insensitive

  b. **Search Inside Content**
  i. SearchResultNode* searchText(string keyword);
  ii. Returns:
  iii. Content title
  iv. Revision number

  c. **Find Content**
  i. ContentMetadata* findContent(string title);

- **Revision Control Features**
  a. Snapshot System
  i. void createSnapshot(string snapshotName);

ii. void restoreSnapshot(string snapshotName);
iii. void listSnapshots();

b. Revision Analysis
   i. void analyzeContentHistory(string title);
   ii. Example Output:
   iii. Revision History for: AI_Article
   iv. Total Revisions: 3
   v. Total Storage Used: 127 bytes
   vi. r1 | timestamp | size
   vii. r2 | timestamp | size (+change)
   viii. r3 | timestamp | size (-change)
   ix. Balance Factor Analysis:
   x. Node (timestamp T1): BF = 1
   xi. Node (timestamp T2): BF = -2

- **Print / Export / Analysis**
  e. Print Revisions
  void printRevisions(string title);

  f. Export Content
  void exportContent(string title, int revisionNo, string outFileName);

  g. System Analysis
  void systemAnalysis();
  Must display:
  i. Total content items
  ii. Total revisions
  iii. Total storage used
  iv. Height of BSTs
  v. Balance factor statistics
  vi. Check if tree is skewed/ complete / perfect etc.

Restrictions

- No STL containers
- No arrays
- No built-in data structures

Memory Management

- Proper destructors required
- No memory leaks

Test cases:

```
#include <gtest/gtest.h>
#include "ContentManagementSystem.h"

class CMS_Test : public ::testing::Test {
protected:
    ContentManagementSystem cms;

    void SetUp() override {
    }

    void TearDown() override {
    }
};
TEST_F(CMS_Test, CreateSingleContent) {
    cms.createContent("Article1", "Hello World");

    auto meta = cms.findContent("Article1");

    ASSERT_NE(meta, nullptr);
    EXPECT_EQ(meta->title, "Article1");
    EXPECT_EQ(meta->totalRevisions, 1);
}
TEST_F(CMS_Test, DuplicateContent) {
    cms.createContent("Article1", "Hello");
    cms.createContent("Article1", "Duplicate");

    auto meta = cms.findContent("Article1");

    EXPECT_EQ(meta->totalRevisions, 1); // should not overwrite
}
TEST_F(CMS_Test, AddRevisions) {
    cms.createContent("Doc1", "v1");

    cms.addRevision("Doc1", "v2");
    cms.addRevision("Doc1", "v3");
```

```
    auto meta = cms.findContent("Doc1");

    ASSERT_NE(meta, nullptr);
    EXPECT_EQ(meta->totalRevisions, 3);
}
TEST_F(CMS_Test, RevisionBSTStructure) {
    cms.createContent("Doc1", "A");

    cms.addRevision("Doc1", "B");
    cms.addRevision("Doc1", "C");

    auto root = cms.findContent("Doc1")->revisionRoot;

    ASSERT_NE(root, nullptr);
    EXPECT_TRUE(root->left == nullptr || root->right != nullptr);
}
TEST_F(CMS_Test, ExactTimestamp) {
    cms.createContent("Doc1", "v1");
    long t1 = cms.getLatestTimestamp("Doc1");

    auto rev = cms.getRevision("Doc1", t1);

    ASSERT_NE(rev, nullptr);
    EXPECT_EQ(rev->text, "v1");
}
TEST_F(CMS_Test, ClosestTimestamp) {
    cms.createContent("Doc1", "v1");
    cms.addRevision("Doc1", "v2");

    long t = cms.getLatestTimestamp("Doc1") - 1;

    auto rev = cms.getRevision("Doc1", t);

    ASSERT_NE(rev, nullptr);
}
TEST_F(CMS_Test, TitleSearch) {
    cms.createContent("AI_Article", "data");
    cms.createContent("ML_Notes", "data");

    SearchResultNode*  results = cms.searchContentTitles("AI");

    ASSERT_NE(results, nullptr);
}
TEST_F(CMS_Test, ContentSearch) {
```

```
    cms.createContent("Doc1", "machine learning");
    cms.addRevision("Doc1", "deep learning");

    auto results = cms.searchText("learning");

    ASSERT_NE(results, nullptr);
    EXPECT_EQ(results->title, "Doc1");
}
TEST_F(CMS_Test, DeleteContent) {
    cms.createContent("Doc1", "text");

    cms.deleteContent("Doc1");

    auto meta = cms.findContent("Doc1");

    EXPECT_EQ(meta, nullptr);
}
TEST_F(CMS_Test, DeleteNonExisting) {
    cms.deleteContent("Ghost");

    SUCCEED(); // should not crash
}
TEST_F(CMS_Test, CreateSnapshot) {
    cms.createContent("Doc1", "v1");

    cms.createSnapshot("S1");

    ASSERT_NE(cms.snapshots, nullptr);
}
TEST_F(CMS_Test, RestoreSnapshot) {
    cms.createContent("Doc1", "v1");
    cms.createSnapshot("S1");

    cms.addRevision("Doc1", "v2");

    cms.restoreSnapshot("S1");

    auto meta = cms.findContent("Doc1");

    EXPECT_EQ(meta->totalRevisions, 1);
}
TEST_F(CMS_Test, BalanceFactorCalculation) {
    cms.createContent("Doc1", "v1");

    for (int i = 0; i < 5; i++)
```

```
        cms.addRevision("Doc1", "v" + std::to_string(i+2));

    auto root = cms.findContent("Doc1")->revisionRoot;

    int bf = cms.getBalanceFactor(root);

    EXPECT_TRUE(bf >= -5 && bf <= 5);
}
TEST_F(CMS_Test, SkewedTree) {
    cms.createContent("Doc1", "v1");

    // Insert increasing timestamps: right skew
    for (int i = 0; i < 10; i++)
        cms.addRevision("Doc1", "v" + std::to_string(i));

    auto root = cms.findContent("Doc1")->revisionRoot;

    int bf = cms.getBalanceFactor(root);
    EXPECT_LT(bf, 0); // right heavy
}
TEST_F(CMS_Test, SystemAnalysis) {
    cms.createContent("A", "v1");
    cms.createContent("B", "v1");

    cms.addRevision("A", "v2");

    cms.systemAnalysis();

    SUCCEED(); // should execute without crash
}
TEST_F(CMS_Test, NoEarlierRevision) {
    cms.createContent("Doc1", "v1");

    long t = cms.getLatestTimestamp("Doc1") - 100;

    auto rev = cms.getRevision("Doc1", t);

    EXPECT_EQ(rev, nullptr);
}
TEST_F(CMS_Test, EmptySearch) {
    auto res = cms.searchText("anything");

    EXPECT_EQ(res, nullptr);
}
TEST_F(CMS_Test, LargeData) {
```

```
    for (int i = 0; i < 50; i++) {
        cms.createContent("Doc" + std::to_string(i), "v1");

        for (int j = 0; j < 10; j++)
            cms.addRevision("Doc" + std::to_string(i), "v" + std::to_string(j));
    }

    SUCCEED(); // performance + stability
}


/* draw tree according to following test case as well, on page and write down its
analysis on page as well. */
TEST_F(CMS_Test, ComplexSystemAnalysis_PrintOnly) {

    cms.createContent("BalancedDoc", "v1");

    cms.addRevision("BalancedDoc", "v2");
    cms.addRevision("BalancedDoc", "v3");
    cms.addRevision("BalancedDoc", "v4");
    cms.addRevision("BalancedDoc", "v5");
    cms.addRevision("BalancedDoc", "v6");
    cms.addRevision("BalancedDoc", "v7");


    cms.createContent("RightSkewedDoc", "v1");

    for (int i = 0; i < 12; i++) {
        cms.addRevision("RightSkewedDoc", "v" + std::to_string(i + 2));
    }

    cms.createContent("HeavyDoc", "v1");

    for (int i = 0; i < 15; i++) {
        cms.addRevision("HeavyDoc", "data_" + std::to_string(i));
    }


    cms.createContent("SingleNodeDoc", "only");

    cms.createContent("MediumDoc", "start");

    cms.addRevision("MediumDoc", "alpha");
    cms.addRevision("MediumDoc", "beta");
    cms.addRevision("MediumDoc", "gamma");
```

```
    cms.addRevision("MediumDoc", "delta");


    for (int i = 0; i < 5; i++) {
        string title = "ExtraDoc_" + std::to_string(i);
        cms.createContent(title, "init");

        for (int j = 0; j < 5; j++) {
            cms.addRevision(title, "rev_" + std::to_string(j));
        }
    }



    cout << "\n===== SYSTEM ANALYSIS OUTPUT =====\n";
    cms.systemAnalysis();
    cout << "==================================\n";


    SUCCEED();
}
```